

Introduction to XML Schema

this slide intentionally left blank

XML Schema

- XML Schema (XSchema) is an XML based language
 - for defining the structure of an XML document
 - schema may also be processed using XML tools

```
<?xml version='1.0' ?>
<xsd:schema xmlns:xsd="http://www.w3.org/2001/XMLSchema">
  <xsd:element name="book">
    <xsd:complexType>
      <xsd:sequence>
        <xsd:element name="title" type="xsd:string"/>
        <xsd:element name="author" type="xsd:string" maxOccurs="5"/>
        <xsd:element name="publisher" type="xsd:string"/>
        <xsd:element name="pubdate" type="xsd:gYearMonth"/>
        <xsd:element name="cost" type="xsd:decimal"/>
      </xsd:sequence>
    </xsd:complexType>
  </xsd:element>
</xsd:schema>
```

```
<!ELEMENT book (title, author+, publisher, pubdate, cost)>
<!ELEMENT title (#PCDATA)>
<!ELEMENT author (#PCDATA)>
<!ELEMENT publisher (#PCDATA)>
<!ELEMENT pubdate (#PCDATA)>
<!ELEMENT cost (#PCDATA)>
```

The XML Schema Definition

- Defines content and structure of target XML
- Individual components defined using XML elements
 - `http://www.w3.org/2001/XMLSchema` namespace
 - namespace prefix `xs` or `xsd` commonly used

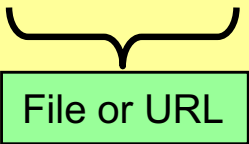
```
<?xml version='1.0' ?>
<xsd:schema xmlns:xsd="http://www.w3.org/2001/XMLSchema">
  <xsd:element name="book">
    ... define structure of element "book" ...
  </xsd:element>
  ... define other components of XML document ...
</xsd:schema>
```

Top level (global)
element may be
root of
target XML

Schemas and Schema Instance Documents

- Document to be validated against schema is known as Schema Instance
- Must inform validator of location of schema
 - details depend on whether namespaces are in use

```
<?xml version='1.0' ?>
<book xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xsi:noNamespaceSchemaLocation="book.xsd">
  <title>Essential XML</title>
  ...
</book>
```



The diagram illustrates the relationship between the schema location attribute in the XML code and its physical location. A curly bracket originates from the value "book.xsd" within the `xsi:noNamespaceSchemaLocation` attribute and points down to a green rectangular box containing the text "File or URL".

- Parser may use other mechanism to locate schema
 - Xerces can use properties

Simple and Complex Types

- Element declarations associate name and type

```
<xsd:element name="title" type="xsd:string"/>
```

- Values of simple types are represented as character strings
 - string, integer, float, boolean, date, etc.
 - XML Schema processors must recognise a set of simple types
- Complex types allow definition of attributes and child elements
- New simple and complex types may be defined
 - more later...

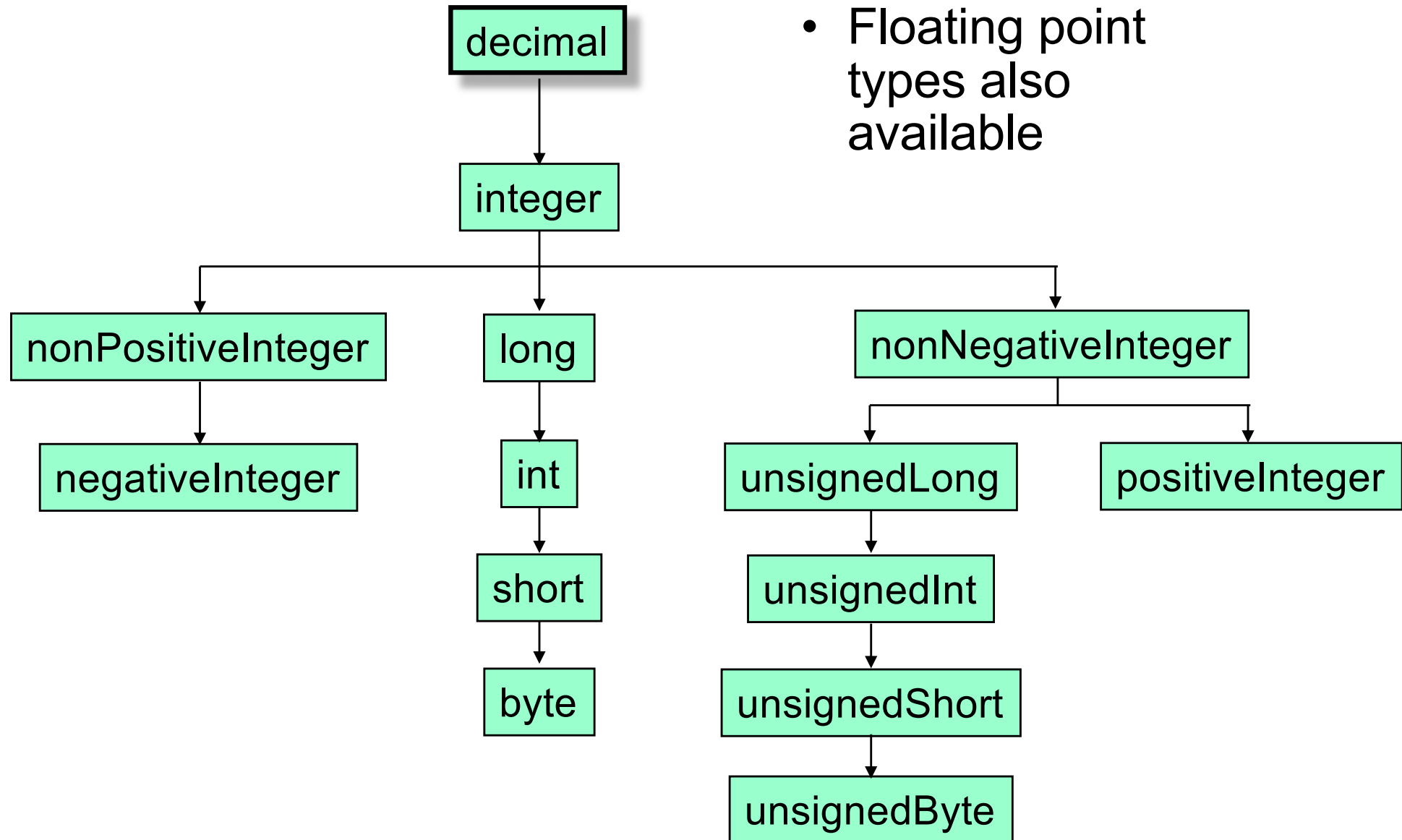
Element Occurrence Count

- Equivalent of ?, +, * operators in DTD
- Indicated by schema attributes
 - minOccurs
 - maxOccurs
 - both default to 1

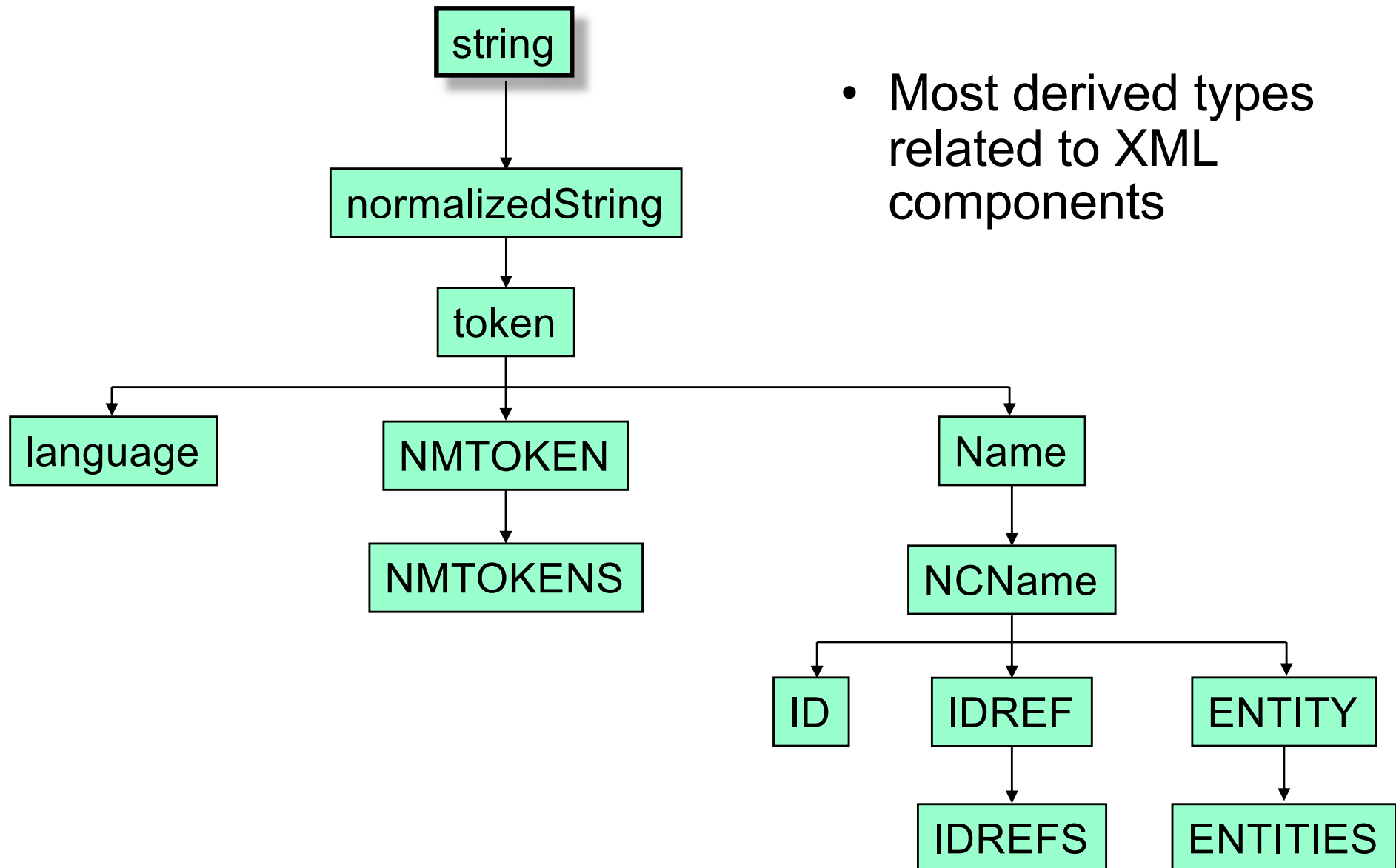
DTD Cardinality Operator	minOccurs Value	maxOccurs Value	No. of Elements
	1	1	Exactly 1
?	0	1	0 or 1
*	0	unbounded	0 or more
+	1	unbounded	1 or more

```
<xsd:element name="author" type="xsd:string" maxOccurs="5"/>
```

Built-in Numeric Simple Types



Built-in String Simple Types



Other Built-in types

- **anySimpleType**
 - "wildcard" type, matching any of the other simple types
- **boolean**
 - values can be "true", "false", "1", "0"
- **date**
 - e.g. "2011-02-04"
- **time**
 - e.g. "12:24:00"
 - may include timezone modifier
- **duration**
 - e.g. "P1M2DT3H5M"
(1 month, 2 days, 3 hours, 5 minutes)
- **float, double**
 - as defined by IEEE

Deriving New Types

- New simple types are derived from a base type
 - may be built-in or derived
- Three means of deriving new simple type
 - restriction
 - list
 - union
 - use "simpleType" element
- Two means of deriving new complex type
 - restriction
 - extension
 - use "complexType" element
- New types may be named or anonymous

New Simple Datatypes

- Derive by restriction
 - operate on base type's Constraining Facets
 - e.g. length, values, pattern match

```
<xsd:simpleType name="US-SSN-Type">  
  <xsd:restriction base="xsd:string">  
    <xsd:length value="11"/>  
  </xsd:restriction>  
</xsd:simpleType>
```

```
<xsd:simpleType name="orderAmountType">  
  <xsd:restriction base="xsd:positiveInteger">  
    <xsd:maxInclusive value="20"/>  
  </xsd:restriction>  
</xsd:simpleType>
```

```
<xsd:simpleType name="MoneyType">  
  <xsd:restriction base="xsd:decimal">  
    <xsd:totalDigits value="7"/>  
    <xsd:fractionDigits value="2"/>  
  </xsd:restriction>  
</xsd:simpleType>
```

Patterns and Enumerations

- Pattern facet can be used to constrain lexical values of a type using Regular Expressions
 - similar to those used in Perl and elsewhere
- New types can also be defined as enumerations

```
<xsd:simpleType name="UK-Phone-Type">  
  <xsd:restriction base="xsd:string">  
    <xsd:pattern value="+44-[1-9]{4}-[0-9]{6}"/>  
  </xsd:restriction>  
</xsd:simpleType>
```

```
<xsd:simpleType name="sizeType">  
  <xsd:restriction base="xsd:string">  
    <xsd:enumeration value="XS"/>  
    <xsd:enumeration value="S"/>  
    <xsd:enumeration value="M"/>  
    <xsd:enumeration value="L"/>  
    <xsd:enumeration value="XL"/>  
  </xsd:restriction>  
</xsd:simpleType>
```


New List Based Simple Types

- List types separate items using whitespace
 - can make string lists troublesome!
- List items can be any simple type
 - including newly defined ones

```
<xsd:simpleType name="authorsType">  
  <xsd:list itemType="xsd:string"/>  
</xsd:simpleType>
```

```
<xsd:simpleType name="specialValueList">  
  <xsd:list>  
    <xsd:simpleType>  
      <xsd:restriction base="xsd:integer">  
        <xsd:maxInclusive value="100"/>  
      </xsd:restriction>  
    </xsd:simpleType>  
  </xsd:list>  
</xsd:simpleType>
```

New Union Based Simple Types

- Allows alternative types for an element
 - like programming language union types

```
<xsd:simpleType name="intOrDate">  
  <xsd:union memberTypes="xsd:integer xsd:date"/>  
</xsd:simpleType>
```

```
<xsd:simpleType name="myUnionType">  
  <xsd:union>  
    <xsd:simpleType>  
      <xsd:restriction base="xsd:integer">  
        <xsd:maxInclusive value="100"/>  
      </xsd:restriction>  
    </xsd:simpleType>  
    <xsd:simpleType>  
      <xsd:restriction base="xsd:date"/>  
    </xsd:simpleType>  
  </xsd:union>  
</xsd:simpleType>
```

Defining a Complex Type

- Child elements must be enclosed within compositor
 - sequence: all elements in specified order
 - choice: one of the elements
 - all: elements in any order but at most one occurrence
- Compositors may be combined
 - e.g. sequence within a choice

```
...  
<xsd:element name="person">  
  <xsd:complexType>  
    <xsd:sequence>  
      <xsd:element name="title" type="xsd:string"/>  
      <xsd:element name="firstName" type="xsd:string"/>  
      <xsd:element name="lastname" type="xsd:string"/>  
    </xsd:sequence>  
  </xsd:complexType>  
</xsd:element>  
...
```

Named Complex Types

- Similar to named simple types

```
<? xml version="1.0" ?>
<xsd:schema xmlns:xsd="http://www.w3.org/2001/XMLSchema">

  <xsd:complexType name="personType">
    <xsd:sequence>
      <xsd:element name="title" type="xsd:string"/>
      <xsd:element name="firstName" type="xsd:string"/>
      <xsd:element name="lastname" type="xsd:string"/>
    </xsd:sequence>
  </xsd:complexType>

  <xsd:element name="person" type="personType" />

</xsd:schema>
```

Attributes

- Declared like elements
 - normally in complex type definition
 - positioned after content description
 - values are always simple types

```
<? xml version="1.0" ?>
<xsd:schema xmlns:xsd="http://www.w3.org/2001/XMLSchema">
  <xsd:complexType name="personType">
    <xsd:sequence>
      <xsd:element name="title" type="xsd:string"/>
      <xsd:element name="firstName" type="xsd:string"/>
      <xsd:element name="lastname" type="xsd:string"/>
    </xsd:sequence>
    <xsd:attribute name="ssn" type="US-SSN-Type"/>
  </xsd:complexType>

  <xsd:element name="person" type="personType" />
</xsd:schema>
```

as defined earlier

Attribute Occurrence and Value Constraints

- No minOccurs and maxOccurs for attributes
- Declaration may have "use" attribute
 - required
 - optional
 - prohibited
- Default value can be specified
 - "default" attribute
- "fixed" attribute allows more rigorous checking
 - cannot have both "fixed" and "default"

```
<attribute name="currency" default="USD"/>
```

Use supplied value,
or "USD" if none

```
<attribute name="currency" fixed="USD"/>
```

Supplied value
must be "USD"

Attributes and Text Only Elements

- Text only elements normally declared using simple type
 - but attributes can only be added with complex type
- Use `<xsd:simpleContent>` schema element

```
<? xml version="1.0" ?>
<xsd:schema xmlns:xsd="http://www.w3.org/2001/XMLSchema">

  <xsd:complexType name="cashType">
    <xsd:simpleContent>
      <xsd:extension base="xsd:decimal">
        <xsd:attribute name="currency" type="xsd:string"/>
      </xsd:extension>
    </xsd:simpleContent>
  </xsd:complexType>

  <xsd:element name="amount" type="cashType" />

</xsd:schema>
```

Empty Elements

- Use complex type
 - implies that content is elements
 - don't specify any elements...

```
<xsd:element name="marker">  
  <xsd:complexType />  
</xsd:element>
```

- Easy to add attributes

```
<xsd:element name="Velocity">  
  <xsd:complexType>  
    <xsd:attribute name="magnitude" type="xsd:integer"/>  
    <xsd:attribute name="direction" type="xsd:positiveInteger"/>  
  </xsd:complexType>  
</xsd:element>
```


XML Schemas and Namespaces

- Namespaces used within XML Schema Recommendation:
- XML Schema Namespace
 - <http://www.w3.org/2001/XMLSchema>
 - defines elements and attributes used in constructing schemas
- XML Schema Datatypes Namespace
 - <http://www.w3.org/2001/XMLSchema-datatypes>
 - defines built in types
 - definitions also included in XML Schema Namespace
- XML Schema Instance Namespace
 - <http://www.w3.org/2001/XMLSchema-instance>
 - defines attributes used in schema instance documents
 - e.g. `noNamespaceSchemaLocation`

Creating Vocabularies in a Namespace

- Schemas are often used to define the structure of elements belonging to a namespace
 - one schema per namespace
- Use `targetNamespace` attribute in schema definition
- Also require namespace definition in schema definition
 - for element names

```
<?xml version="1.0" ?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema"
            targetNamespace="http://www.gb.com/people"
            xmlns="http://www.gb.com/people"
            elementFormDefault="qualified">

    <xs:element ... >
</xs:schema>
```

Qualified and Unqualified Elements

- All global elements belong to the target namespace
 - "qualified"
 - local elements may or may not
 - attributes may or may not belong to target namespace
- Use form attribute to specify qualified or not
 - `elementFormDefault` attribute specifies default for elements
 - `attributeFormDefault` attribute specifies default for attributes
 - both default to "unqualified"

```
<xs:element name="el1" form="qualified" ... />
<xs:element name="el2" form="unqualified" ... />
<xs:attribute name="attr1" form="qualified" ... />
<xs:attribute name="attr2" form="unqualified" ... />
```

Example – The Book Schema

```
<?xml version="1.0"?>
<xsd:schema xmlns:xsd="http://www.w3.org/2001/XMLSchema"
            xmlns="http://www.gb.com/book"
            targetNamespace="http://www.gb.com/book"
            elementFormDefault="qualified">
  <xsd:element name="book">
    <xsd:complexType>
      <xsd:sequence>
        <xsd:element name="title" type="xsd:string"/>
        <xsd:element name="author" type="xsd:string"/>
        <xsd:element name="publisher" type="xsd:string"/>
        <xsd:element name="pubdate" type="xsd:anySimpleType"/>
        <xsd:element name="cost" type="xsd:decimal"/>
      </xsd:sequence>
    </xsd:complexType>
  </xsd:element>
</xsd:schema>
```

← All defined elements belong to the target namespace

The Schema Instance Document

- Schema location indicated by attribute
 - or features as before
- Different syntax

```
<?xml version='1.0' ?>
<book xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xmlns=http://www.gb.com/book"
      xsi:schemaLocation="http://www.gb.com/book book.xsd" >

  <title>Essential XML</title>
  <author>
    <first>Don</first>
    <last>Box</last>
  </author>
  <publisher>Addison Wesley</publisher>
  <pubdate>2000</pubdate>
  <cost>23.50</cost>
</book>
```

Using Other Namespaces

- Schema for one namespace may use definitions from another

```
<?xml version="1.0"?>
<xsd:schema xmlns:xsd="http://www.w3.org/2001/XMLSchema"
            targetNamespace="http://www.gb.com/book"
            elementFormDefault="qualified"
            xmlns:people="http://www.gb.com/people"
            xmlns="http://www.gb.com/book">

    <xsd:import namespace="http://www.gb.com/people"
                schemaLocation="people.xsd" />

    <xsd:element name="book">
        <xsd:complexType>
            <xsd:sequence>
                <xsd:element name="title" type="xsd:string"/>
                <xsd:element name="author" type="people:personType"/>
                ...
            </xsd:sequence>
        </xsd:complexType>
    </xsd:element>
</xsd:schema>
```